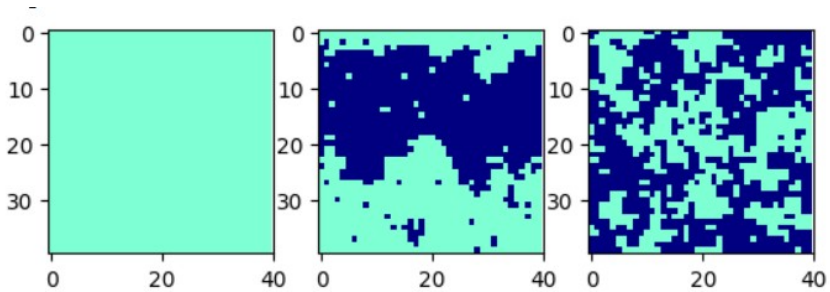
Tutorial Five: Ising Model, Classified

What would you call a group of people who don't know about fish eggs that have discussions about figments of imaginations together?

1 Convolutional Neural Network (CNN)

As we have learned in the lecture, CNN is especially powerful in learning the features of images. The following are some sample images we have seen in tutorial 1.



In this tutorial, we will train a CNN to classify the Ising model and detect the phase transition at the critical temperature. If this works, we can use it to detect the critical temperature in other models that do not have an analytical solution (Most models do not admit analytical solutions. You can ask Pedro about integrability next block.)

First we will read in the tutorial 1 data as before (make sure they are in the same folder). Then we will convert the data to the correct datatype for pytorch to use in a CNN. (float32 for features, even if they are just -1 or 1 for us, because they will multiply the filters which are matrices with real numbers. And long for labels, which remain integer (as an index should be), but pytorch likes to use all labels long even if we only have two labels).

And then we will shape our feature in the shape of

$$(N_{sample}, Channel, Height, Width) \tag{1}$$

For the data we have, the height and width are both 40 since they form a 40×40 2d Ising lattice simulation. We have $16 \times 10,000 = 160,000$ samples (16 different temperatures and 10,000 data each).

a) If grayscale is 1 channel, color RGB is 3 channel, how many channels do we have as input?

   Reshape our features and we are ready to hand over to a CNN. But wait, CNN costs a lot of memory (if you have time, try to do a quick estimate how much memory we need for each data.) with the weights in the convolutional layers and the gradients and memory cost from the optimizer. It is not memory friendly to hand in all 160,000($\times 40 \times 40$) data to a CNN. The kernel is very likely to crush (sometimes saying it is memory issue sometimes not.). The next block of code split our data into train/test sets as before and create so-called `DataLoader`s. DataLoaders are iterable objects (that stores references to the data[1] but not the data themselves) that when we iterate over them later during training or testing, a smaller batch of samples (in our case, 32) are (randomly if shuffled) selected and go through the network until all the training/testing data are used[2].

   Now we are ready to define the CNN architecture.

   For some reason, a hungry caterpillar[3] has crawled through your CNN and devoured some parts of the architecture. As a result, some numbers have been replaced by question marks "??". Luckily, you know CNN well enough and you are able to fill in the correct numbers to build your CNN.

b) Fix the `def __init__(self):` function in the `class IsingCNN(nn.Module):`.

[1] At the start of each epoch, DataLoader will generate a random permutation of indices. No data is repeated within each epoch and the data together in a batch in one epoch may go into different batches in another epoch.

[2] Conveniently, our number of samples is divisble by the batch size, otherwise you can use `drop_last = False` to force it.

[3]

Now we are really ready to train CNN. Notice that we only need about 10 epochs! We will use fancy GPU for CNN. This is indicated by the `device =...` line.

The criterion we are using is `CrossEntropyLoss()` instead of the usual `MSE()`.

c) How does `CrossEntropyLoss()` work? How is it defined for each sample?

For CNN, since we are feeding training samples in small batches, we need to iterate over those iterables (DataLoader). For each iteration, the CNN model will predict something (after going through some convolutional layers, some pooling layers, some fully connected layers), but then it works just like a usual NN: compute loss, back propagation gradient descent train the weights, repeat. We accumulate the loss through batches and observe the total loss.

d) For every epoch, how many samples have the CNN seen while training?

Now it is time to evaluate our model. Let us directly go to the most thorough way: confusion matrix. There is no embedded confusion matrix function but no big deal, we will go through each test sample (batches rather), and classify them as TP, TN, FN, FP ourselves.

The CNN output is a logit result for both classes. In other words, not a 0 or 1 label yet. The logits are something that we can use to compute the probability. For example, we have two classes 0 and 1. If the sample i has logits $z_0^{(i)}$ and $z_1^{(i)}$, the probability for it[4] to be in class 0 is

$$p_0^{(i)} = \frac{e^{z_0^{(i)}}}{\sum_{\alpha=0,1} e^{z_\alpha^{(i)}}}. \tag{2}$$

We will use `argmax` to return the index of the larger logits (column wise) to make our prediction, which is a 0 or 1 to compare with the true labels. We will then tally up the TP, TN, FP, FN cases.

e) Now we can plot our confusion matrix and compute the accuracy (you need to write a code for this).

First we run it through the training data to find the accuracy score on the training data.

[4] This strongly mimics a partition function, and logits are like $-\beta E$. Thus large logits means lower energy and hence high probability.

This is a very high accuracy score for training data.

f) We should do this again with the testing data.

It is still a very impressive testing score. But we could be fooled, after all, the overall accuracy does not tell much as we learned from logistic regression. The true teller is how it behaves around the critical temperature. We will come back to this.

Now a very cool thing we can do is ask our CNN to detect the phase transition for us.

First we need to make a temperature array (which you probably did in all different ways in your homework). Then we will plot the probability of the sample[5] is in the ordered phase as a function of temperature.

g) Before you plot it, can you guess what kind of function do we expect?

Oh là là. Isn't this cool?

h) Now pick a smaller sample to zoom in around the critical temperature to see how CNN perform and compare with SVM in your homework.

2 Transformers

Transformers[6] are cool.

Now we will use transformers to classify Ising model. We import libraries, load the data and make dataloaders just like before. There will be some changes in the library we import and the batch_size in the dataloaders, so please do rerun these cells even they look similar like before. (Also it might be a good time to restart the

[5] We use the function `softmax` to convert logits to probabilities.

[6]

kernel because the codes always work better after you "turn it off and on again"[7].).

Now let us look into the architecture of the transformers. It is kind of like taking the legos apart and build something else so transforming indeed. Consider some of the following questions a reading comprehension :)

a) By reading the code about the dataloaders, what is the shape of the input that goes to the patch embedding process in our case of the Ising model?

b) In the `forward` function of `PatchEmbedding`, what is the shape of the `patches` as an input for `embeddings`?

c) What is the dimension of the `embeddings` after `self.proj` is applied to `patches`?

   Now let us move on to the transformer class.

d) What is the shape of the input of the function `self.patch_embed` in `forward`? You might also want to read the following train block.

e) What is the shape after going through `self.patch_embed`? (considering what needs to go as an input to the next step could help, this applies to all these four questions.)

f) What is the shape after going through `self.transformer`?

g) What is the shape after going through `x.mean(dim=1)`?

h) What is the shape after going through `self.fc`?

   Now train the model.

---

[7]

i) It takes some time to train transformers, here is some knowledge from the future: the total loss after the five epochs are typically of order hundreds. And the accuracy is often quite good. How do you reconcile this seemingly large loss and the incredible accurate result? *Hint: What is the loss if you unfortunately predict a fifty-fifty probability for one of these near critical samples?*

j) The output of the transformers are again logits as in CNN. Write codes to evaluate as before and get the transformers to predict the phase transition temperature. Comment on what you observe. If you are not happy with the result, especially seeing oscillations in the loss (or not, since we did not have a random seed, the results are somewhat random), you can tune the learning-rate to get a better result. Do you tune the learning-rate bigger or smaller?

Convo lusion no knew roe network.